eBPF'26 · 개별연구 7주차 논문 리뷰

Netkit

Specializing Linux Packet Delivery for Container Networks

Daniel Borkmann et al.

26.10.15 박수현

목차

1	배경 및 문제 정의	veth 경로 · 불필요한 backlog
2	제안 기법	redirect_peer · netkit 디바이스
3	평가	TCP_RR · TCP_CRR · CPU 사용량
4	추가 실험	bpftrace로 backlog 관측

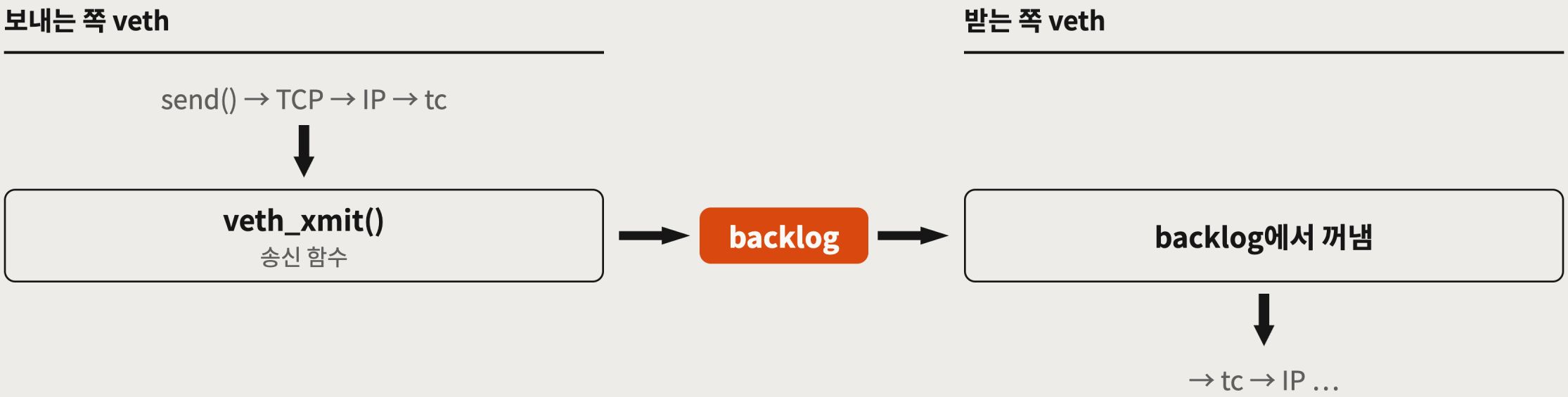
1. 배경 및 문제 정의: 컨테이너 간 통신

호스트 물리 서버 1대 · 커널 1개



송수신 전환, tx → rx 두 번 발생

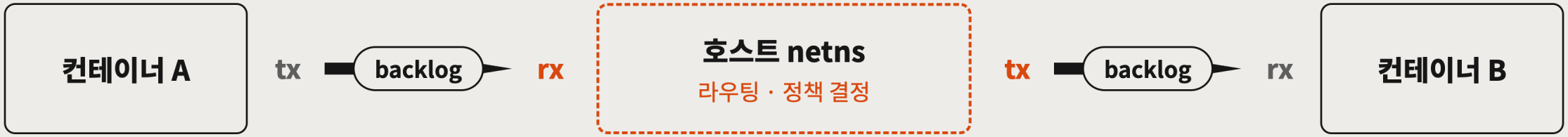
1. 배경 및 문제 정의: veth



veth: 기존 디바이스, NIC 역할, 쌍으로 존재

backlog: 하드웨어 큐 역할 → 송·수신 스택 분리

1. 배경 및 문제 정의: backlog 중첩

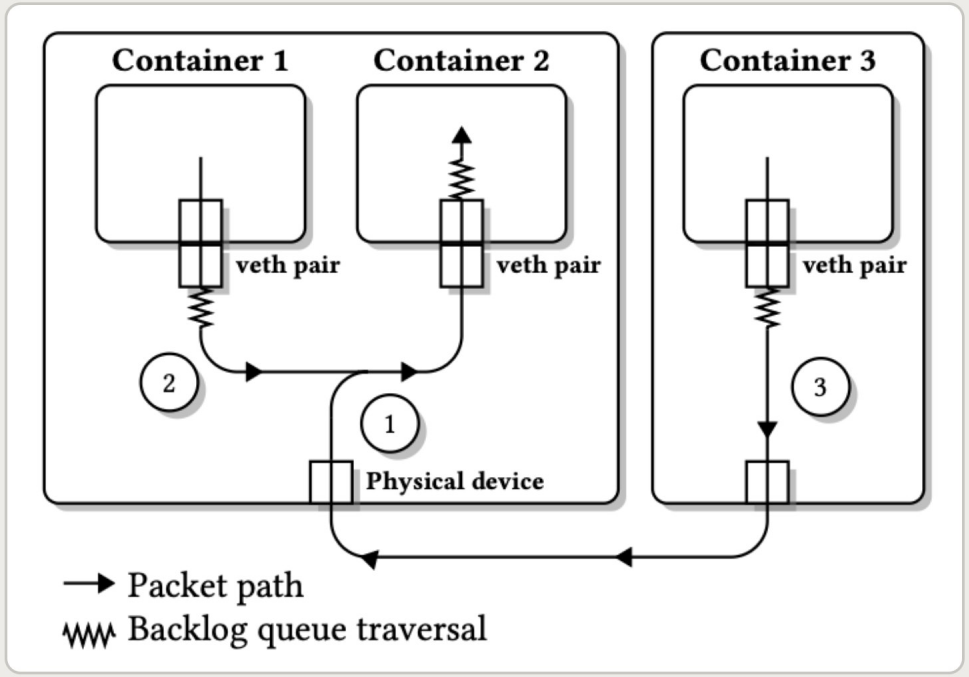


불필요한 것: 라우팅이 아닌 호스트 rx → tx 경유 비용

라우팅·정책 결정은 필요, backlog 재진입 + 호스트 스택 재통과가 낭비

1. 배경 및 문제 정의: 경로별 backlog

컨테이너 간 패킷 경로 (Fig. 2)



같은 머신 · 경로 ②

한 머신 안에서 backlog 2번

실제 필요한 전환 1번 → **1번 불필요**

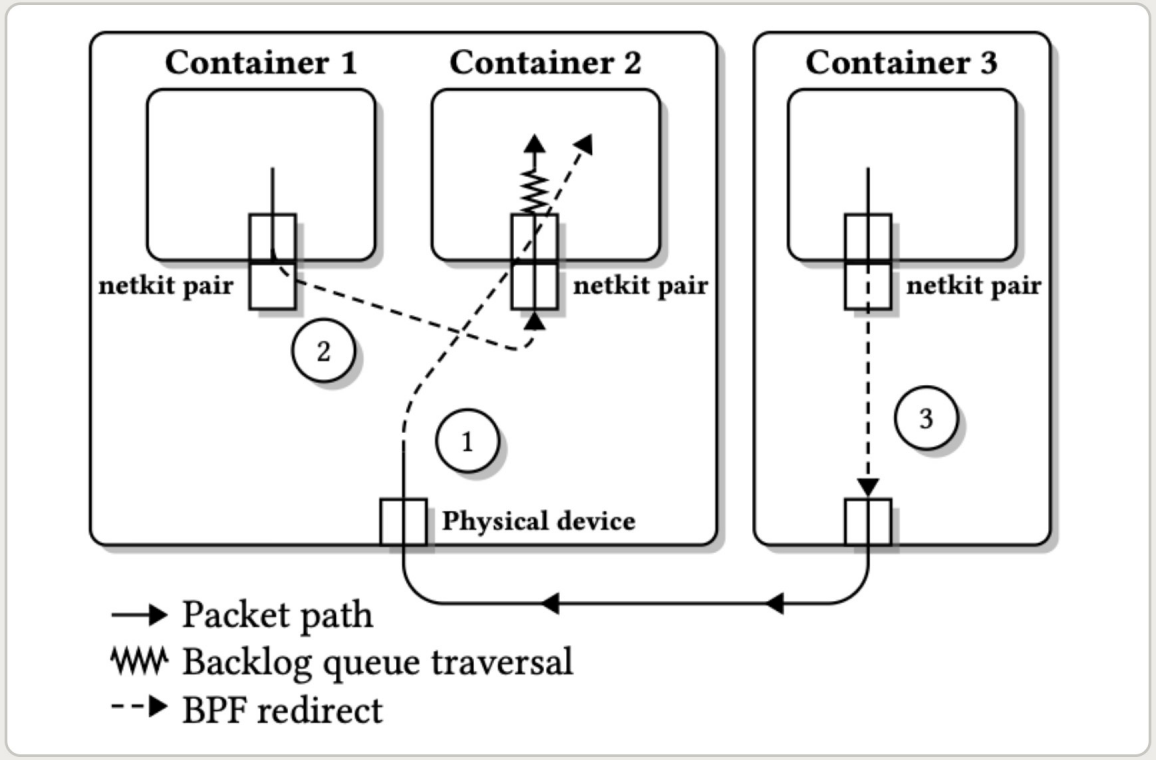
다른 머신 · 경로 ③ + ①

실제 전환은 NIC 경계에서 발생

→ **2번 모두 불필요**

2. 제안 기법: netkit 적용 후 경로

적용 후 경로 (Fig. 4)



경로 ① 외부 → 컨테이너

BPF redirect로 바로 진입 · **backlog 0번**

경로 ③ 컨테이너 → 외부

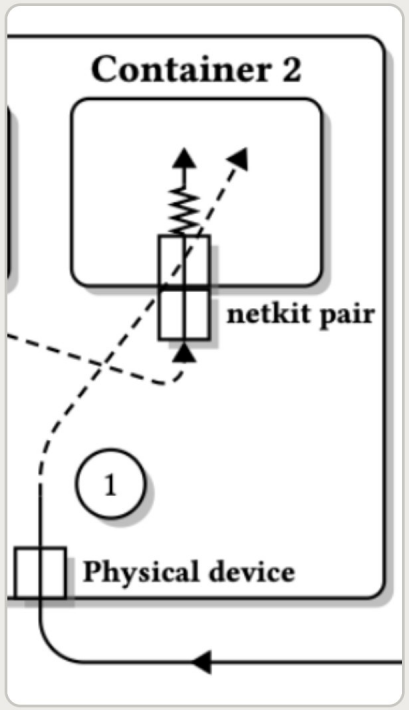
BPF redirect로 바로 송신 · **backlog 0번**

경로 ② 컨테이너 → 컨테이너

redirect 후 수신 측 앞에서 · backlog 1번

2. 제안 기법: 수신 측 bpf_redirect_peer()

Fig. 4 · 경로 ①



기존

NIC 수신 → 호스트 veth 송신 → **backlog** → 컨테이너

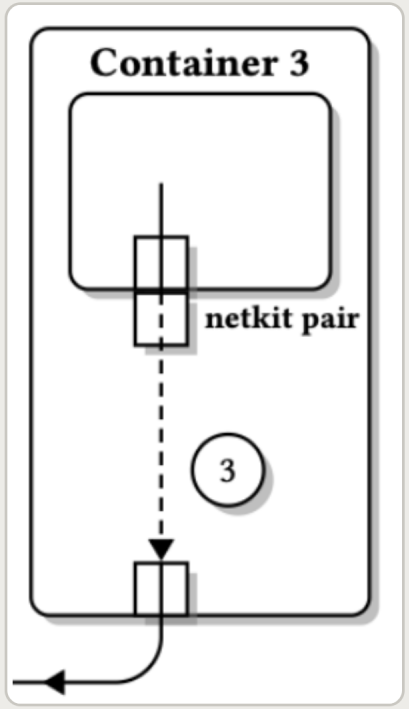
netkit

NIC 수신 → **호스트 netns** tc-bpf → **bpf_redirect_peer()** skb→dev 교체 = netns 교체 → **컨테이너** IP → TCP → 소켓

dev만 교체 → 수신 함수 재실행, backlog 없음

2. 제안 기법: 송신 측 netkit 디바이스

Fig. 4 · 경로 ③



문제점

① 컨테이너 내부에서 외부 NIC 조회 불가

② egress에서 netns를 전환하는 BPF 헬퍼 부재

netkit 디바이스 primary(호스트 쪽) · peer(컨테이너 쪽) 쌍

send → TCP → IP → **netkit_xmit()** skb→dev = primary → **BPF redirect** skb→dev = NIC

송신 시점 dev 교체 → NIC로 직행, backlog 없음

2. 제안 기법: veth와의 차이

① 송신 경로에서 netns 전환

veth: backlog 경유 후 전환 · netkit: netkit_xmit()에서 바로 전환

② 비대칭 구조 → 호스트 관리, 격리 강화

veth

양 끝 대칭

컨테이너가 자기 쪽 BPF 부착·제거 가능

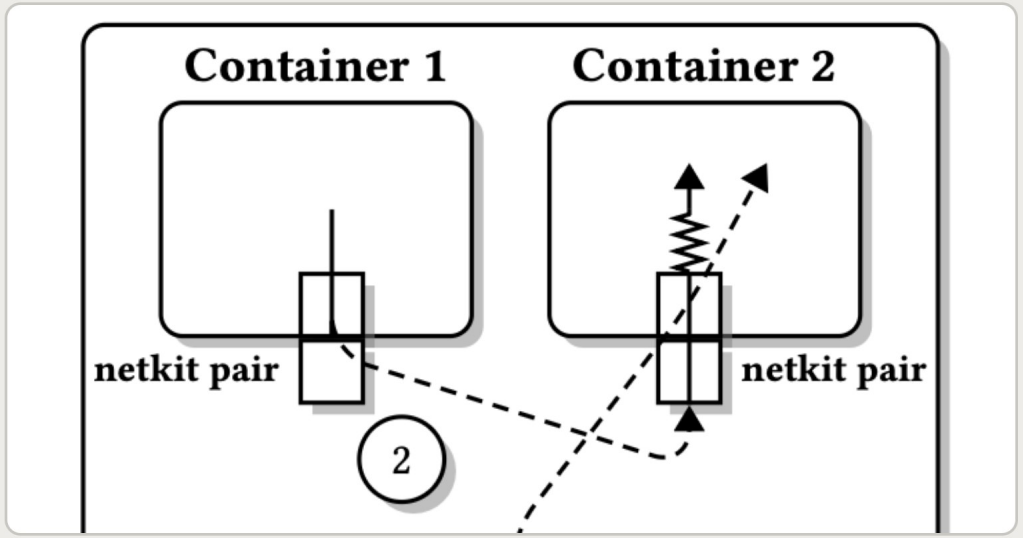
netkit

BPF 등록은 primary로만

컨테이너 netns에서 **접근 불가**

2. 제안 기법: 같은 머신 컨테이너 간

Fig. 4 · 경로 ②



veth backlog 2번

1 송신 → backlog → 호스트 → backlog → 2 수신

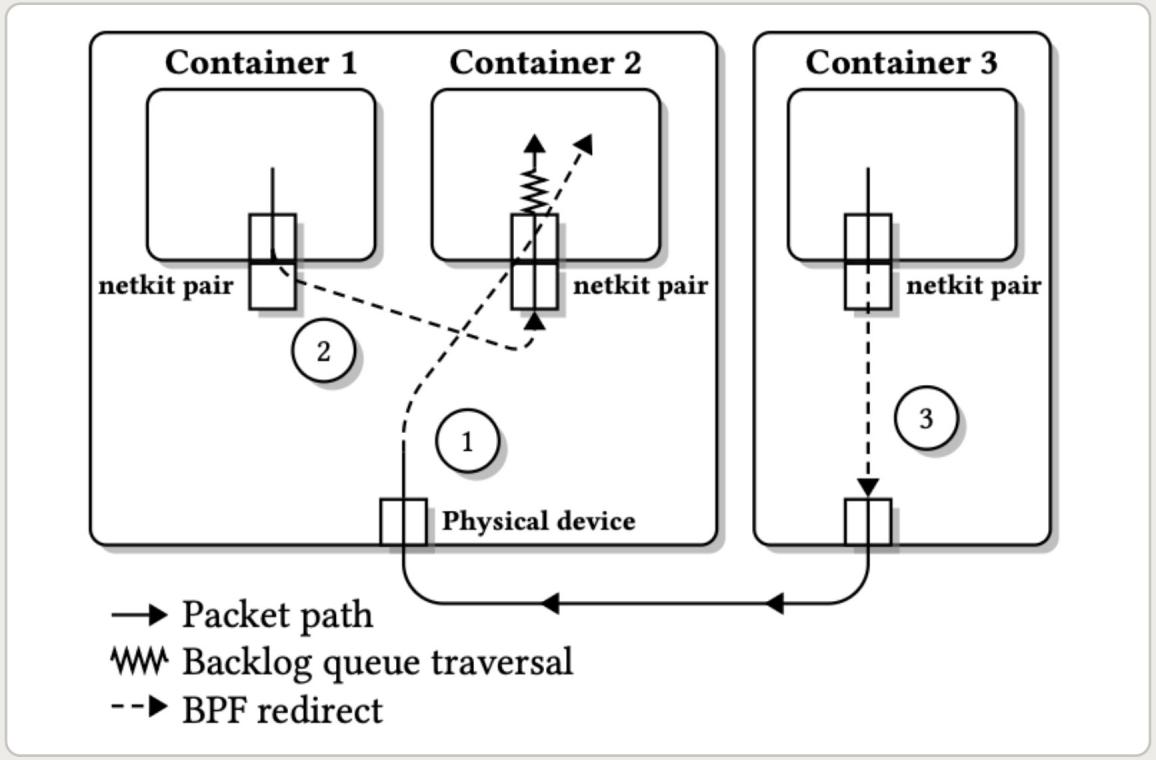
netkit backlog 1번

1 송신 → 1 송신 netkit BPF → 2 송신 → backlog → 2 수신

남은 1번 = 2 송신 → 2 수신, 실제 전환은 제거 불가

2. 제안 기법: 정리

적용 후 경로 (Fig. 4)



경로 ① 외부 → 컨테이너

BPF redirect로 바로 진입 · **backlog 0번**

경로 ③ 컨테이너 → 외부

BPF redirect로 바로 송신 · **backlog 0번**

경로 ② 컨테이너 → 컨테이너

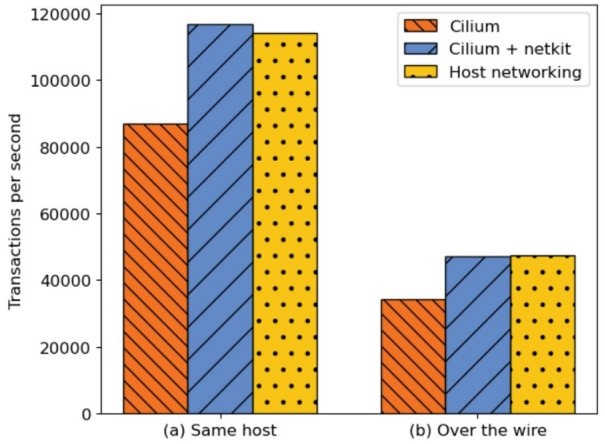
redirect 후 수신 측 앞에서 · backlog 1번

3. 평가: 처리량

Cilium veth + BPF · **Cilium+netkit** veth를 netkit으로 교체 · **Host** 호스트 netns 직접 사용

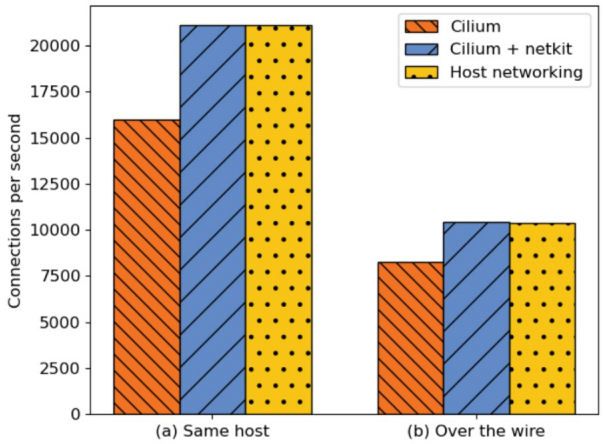
TCP_RR · 초당 트랜잭션 (Fig. 5)

연결 1번 유지 · 요청-응답 반복



TCP_CRR · 초당 연결 수 (Fig. 6)

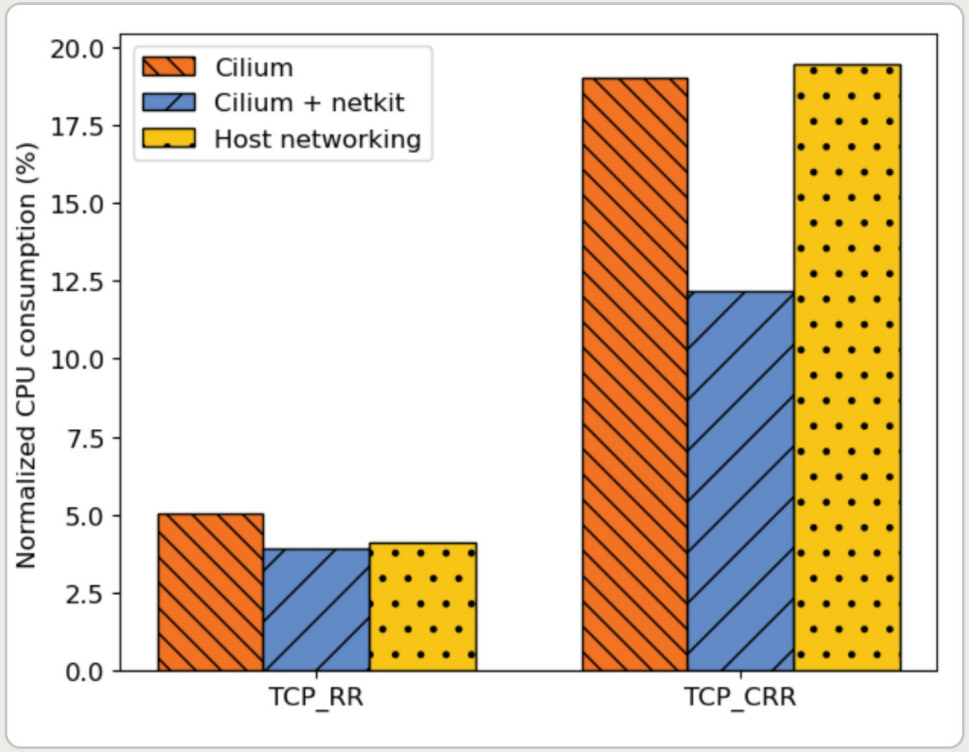
매번 연결 생성 → 요청-응답 → 종료 반복



veth 대비 크게 향상, Host와는 오차 범위 내 동률

3. 평가: CPU 사용량

수신 측 정규화 CPU (Fig. 7, 다른 머신)



conntrack: 연결마다 상태를 기록하는 netfilter 테이블 · 연결 생성·삭제 비용 큼

	Cilium	+netkit	Host
conntrack 우회	O	O	X
backlog 제거	X	O	O

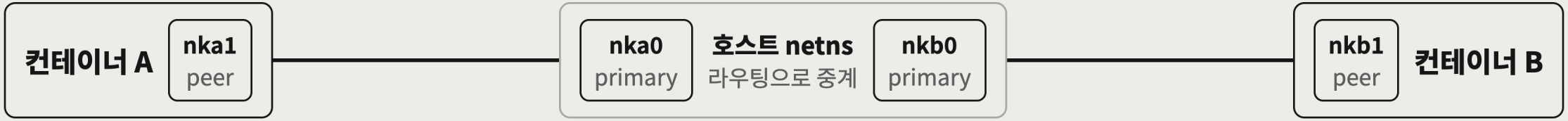
RR · 연결 1번 → conntrack 영향 작음

CRR · 연결 반복 → conntrack 영향 큼

CRR: 두 이점 모두 가진 netkit만 CPU 최저

4. 추가 실험: 실험 구성

호스트 커널 1개 · netkit 쌍 2개 (nka0–nka1, nkb0–nkb1) · 컨테이너 = netns



트래픽

A → B ping (요청 + 응답 왕복)

관측

bpftrace로 **netif_rx** 횟수 집계

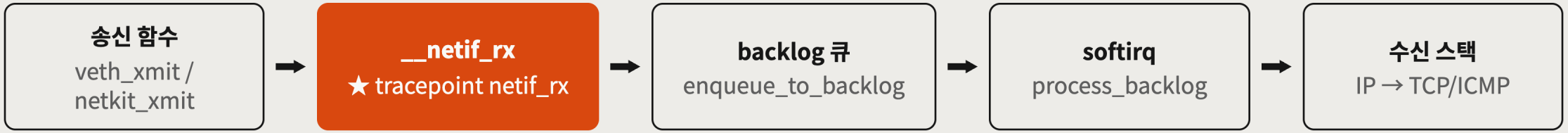
디바이스별 = **backlog 진입 횟수**

비교 조건

① BPF 없음 (호스트 라우팅으로 중계)

② peer 슬롯에 redirect BPF

4. 추가 실험: 관측 지점 netif_rx



무엇을 세나

backlog에 넣기 직전 1번 발생

→ 횟수 = **backlog 진입 횟수**

디바이스 이름 = 받는 쪽

skb→dev를 상대 끝으로 바꾼 뒤 호출

→ 찍히는 이름 = 패킷을 **받는** 디바이스

4. 추가 실험: backlog 진입 횟수

① BPF 없음

```
psh@psh-MS-7E28:~/netkit-lab$ sudo bpftrace -e 'tracepoint:net
:netif_rx { @[str(args->name)] = count(); }'
Attached 1 probe
^C

@[enp6s0]: 1
@[nkb0]: 10
@[nka0]: 10
@[nkb1]: 10
@[nka1]: 10
@[lo]: 544
```

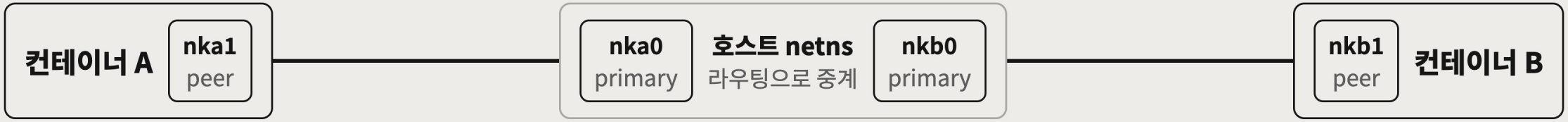
nka0 10 · nkb0 10 · nka1 10 · nkb1 10 → **총 40**

② peer 슬롯 redirect BPF

```
psh@psh-MS-7E28:~/netkit-lab$ sudo bpftrace -e 'tracepoint:net:netif_rx {
 @[str(args->name)] = count(); }'
[sudo: authenticate] 비밀번호:
Attached 1 probe
^C

@[nkb1]: 10
@[nka1]: 10
```

nka1 10 · nkb1 10 → **총 20**



redirect BPF: 호스트 쪽 nka0 · nkb0 backlog 진입 0 → 총 40 → 20

코드 · github.com/clapppp/ebpf-net-experiments